

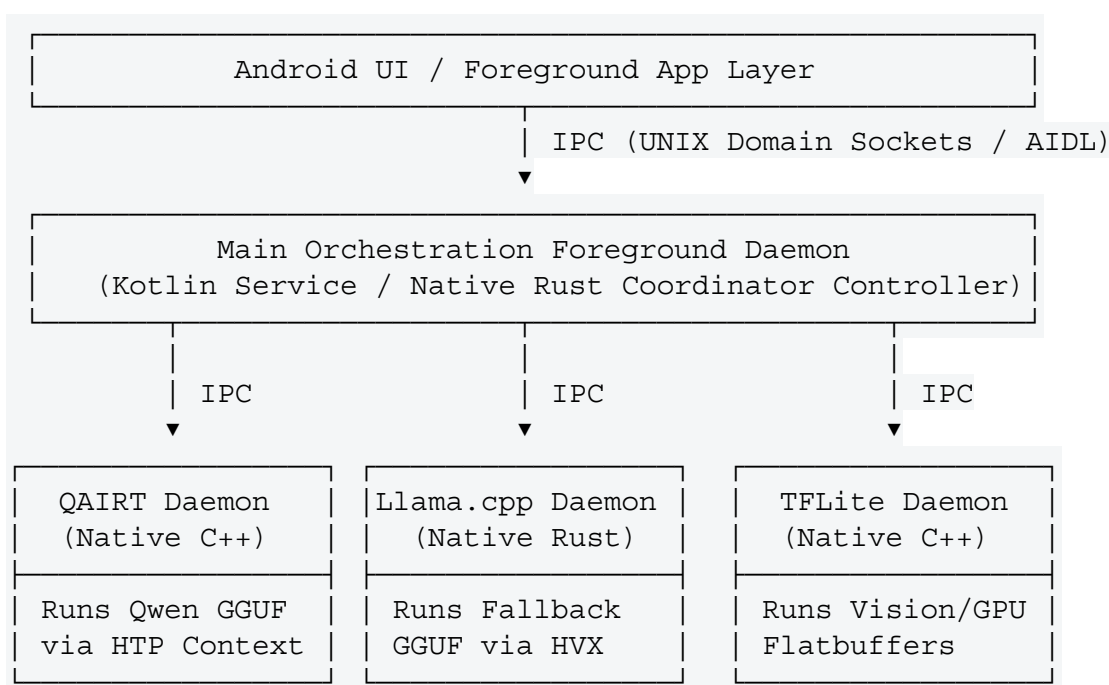
To orchestrate multiple frontends, backends, and multi-language models (TFLite, GGUF, QAIRT) running as interconnected **system daemons (background processes)** on Android, you must step away from standard in-app architecture.

Android enforces strict application sandboxing and lifecycle management. To keep background daemons running persistently, isolated from each other, and communicating with low latency, you must implement **Native C++/Rust Daemons managed by an Android Foreground Orchestration Service** utilizing high-speed **Inter-Process Communication (IPC)**.

The production architecture to tie these daemons together across Kotlin, Java, Rust, and C++ is detailed below.

---

## 1. The Distributed System Daemon Architecture



---

## 2. High-Speed IPC Daemon Communication Protocol

Do not use standard network sockets (localhost:8080) or local HTTP servers. They introduce HTTP parsing overhead, loopback latency, and expose security vulnerabilities across apps.

Instead, use **UNIX Domain Sockets (UDS)** or **Android Shared Memory (Ashmem /**

**memfd\_create**) managed natively in your Rust/C++ daemons. This passes multi-gigabyte Qwen 3.5 tensor outputs across processes at the speed of RAM memory mapping.

## The Protocol Message Structure (Rust C-ABI Interface)

Define a uniform serialization protocol (like FlatBuffers or lightweight protocol buffers) to stream messages between daemons without overhead:

```
// protocol.rs - Shared across all native daemons
#[repr(C)]
pub enum DaemonCommand {
    LoadModel { backend_id: u8, path_ptr: *const c_char },
    Inference { prompt_ptr: *const c_char },
    UnloadModel,
    KillDaemon,
}

#[repr(C)]
pub struct DaemonResponse {
    pub status_code: i32,
    pub token_buffer_fd: i32, // File Descriptor pointing to Shared
    // Memory region
}
```

---

## 3. Implementing a Persistent Native Daemon in Rust

Each engine (QAI RT, Llama.cpp, TFLite) runs inside its own standalone executable looping over a local UNIX socket. Here is how your GGUF/QAI RT native daemon listens for orchestrator commands:

```
// qairt_daemon_main.rs
use std::os::unix::net::UnixListener;
use std::io::{Read, Write};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // 1. Establish an isolated domain socket path inside the app's
    // internal cache
    let socket_path =
        "/data/data/com.example.aiorchestrator/files/qairt_daemon.sock";
    let _ = std::fs::remove_file(socket_path);
```

```

let listener = UnixListener::bind(socket_path)?;
println!("QAIRT Native HTP Daemon Active and Listening...");

for stream in listener.incoming() {
    match stream {
        Ok(mut stream) => {
            let mut buffer = [0; 1024];
            stream.read(&mut buffer)?;

            // 2. Parse command byte and coordinate the
hardware execution
            let command_byte = buffer[0];
            match command_byte {
                1 => { // Load Qwen GGUF mapped into QAIRT HTP
Context
                    initialize_qairt_htp_engine();
                },
                2 => { // Run token processing loop
                    stream.write_all(b"Token Stream
Starting...").unwrap();
                },
                _ => break,
            }
        }
        Err(_) => {}
    }
}
Ok(())
}

```

---

## 4. Managing Daemon Lifecycles in Kotlin

On Android, pure native background processes will be aggressively terminated by the Low Memory Killer (LMK). To prevent this, your Kotlin layer must run a **Sticky Foreground Service** that spawns, monitors, and restarts the native daemons via runtime process shells.

```

// AiOrchestratorService.kt
class AiOrchestratorService : Service() {

    private var qairtProcess: Process? = null
    private var llamacppProcess: Process? = null

    override fun onStartCommand(intent: Intent?, flags: Int,

```

```

startId: Int): Int {
    // 1. Promote to Foreground to shield the background
    daemons from Android OS eviction
    startForeground(NOTIFICATION_ID,
createSystemNotification())

    // 2. Fork the compiled native C++/Rust daemons inside the
    app storage binary path
    val nativeBinDir = applicationInfo.nativeLibraryDir

    qairtProcess =
Runtime.getRuntime().exec("$nativeBinDir/libqairt_daemon.so")
    llamacppProcess =
Runtime.getRuntime().exec("$nativeBinDir/libllamacpp_daemon.so")

    return START_STICKY // Forces system to recreate the
orchestrator if killed
}

override fun onDestroy() {
    // Send shutdown signals to the sockets gracefully before
    closing up shop
    qairtProcess?.destroy()
    llamacppProcess?.destroy()
    super.onDestroy()
}

override fun onBind(intent: Intent?): IBinder? = null
}

```

---

## 5. Automated Multi-Binary Compilation in GitHub CI

To generate multiple independent daemon executables (libqairt\_daemon.so, libllamacpp\_daemon.so) instead of just a single massive block, configure your Rust Cargo.toml and GitHub CI pipeline to build independent binary outputs targeted to the Android linker.

### Update your Cargo.toml

```

[[bin]]
name = "qairt_daemon"
path = "src/bin/qairt_daemon_main.rs"

```

```
[[bin]]
name = "llamacpp_daemon"
path = "src/bin/llamacpp_daemon_main.rs"
```

## GitHub Actions Workflow Assembly Configuration

```
# Inside .github/workflows/daemons-compiler.yml
- name: Compile and Package Daemon Cluster
  run: |
    # Build all binaries optimized for aarch64 Android NPU
    architectures
    cargo ndk -t aarch64-linux-android -p 26 -- build --release
    --bins

    # Android looks for .so files inside jniLibs, even if they are
    executable binaries
    mkdir -p ./android/app/src/main/jniLibs/arm64-v8a
    cp ./target/aarch64-linux-android/release/qairt_daemon
    ./android/app/src/main/jniLibs/arm64-v8a/libqairt_daemon.so
    cp ./target/aarch64-linux-android/release/llamacpp_daemon
    ./android/app/src/main/jniLibs/arm64-v8a/libllamacpp_daemon.so
```

---

## Daemon Architecture Performance Metrics

| Architectural Element            | Strategic Priority                  | Mission Result                                                         |
|----------------------------------|-------------------------------------|------------------------------------------------------------------------|
| <b>UNIX Domain Sockets</b>       | Direct memory lane links.           | Sidesteps network virtualization stack lag completely.                 |
| <b>Sticky Foreground Service</b> | Claims priority status from kernel. | Prevents Android from purging memory-heavy 9B model daemons.           |
| <b>Isolated Executables</b>      | Crashes are decoupled.              | A fault in the TFLite GPU pipeline won't drop the running GGUF engine. |

If you'd like, let me know:

- If you plan to use **FlatBuffers** or raw **Protobuf** payloads to handle your inter-daemon

structural messages.

- If you need a specific **process watch-dog** implementation in Rust to instantly restart down-stream daemons if one exhausts the device VRAM.

I can write out the specific buffer-sharing code utilizing Android's native ashmem structures!